```
// counter.store.ts

import { computed } from '@angular/core';
import {
  signalStore,
  withState,
  withComputed,
  withMethods,
  patchState
} from '@ngrx/signals';

export interface CounterState {
  count: number;
}
const initialState: CounterState = {
  count: 0
};

export const CounterStore = signalStore(
  { providedIn: 'root' },
  withState(initialState),
  withComputed((store) => ({
    doubleCount: computed(() => store.count() * 2)
  })),
  withMethods((store) => ({
    increment() {
      patchState(store, {
        count: store.count() + 1
      });
    },
    decrement() {
      patchState(store, {
        count: store.count() - 1
      });
    },
    reset() {
      patchState(store, initialState);
    }
  }))
);
```

```
import { createReducer, on } from '@ngrx/store';
import * as CounterActions from './counter.actions';

export const initialState = 0;


//Selector
import { createFeatureSelector } from '@ngrx/store';

export const selectCount =
createFeatureSelector<number>('count');

// Action
import { createAction } from '@ngrx/store';

export const increment = createAction('[Counter] Increment');
export const decrement = createAction('[Counter] Decrement');
export const reset = createAction('[Counter] Reset');

// Reducer
export const counterReducer = createReducer(
initialState,

on(increment, (state) => state + 1),

on(decrement, (state) => state - 1),

on(reset, () => 0)
);
```

```
import { Injectable } from '@angular/core';
import { Observable, timer } from 'rxjs';
import {retryWhen,delayWhen,  tap, share } from 'rxjs/operators';
import { webSocket,  WebSocketSubject
} from 'rxjs/webSocket';

@Injectable({ providedIn: 'root' })
export class WsService {

  private socket$!: WebSocketSubject<any>;

  connect(url: string): Observable<any> {

    this.socket$ = webSocket({
      url,
      openObserver: {
        next: () => console.log('Connected')
      },
      closeObserver: {
        next: () => console.log('Disconnected')
      }
    });
```

```
  return this.socket$.pipe(
      retryWhen(errors =>
        errors.pipe(
          tap(() => console.log('Retrying...')),
          delayWhen(() => timer(3000))
        )
      ),

      share()
    );
  }

  send(message: any): void {
    this.socket$.next(message);
  }

  disconnect(): void {
    this.socket$.complete();
  }
}
```
